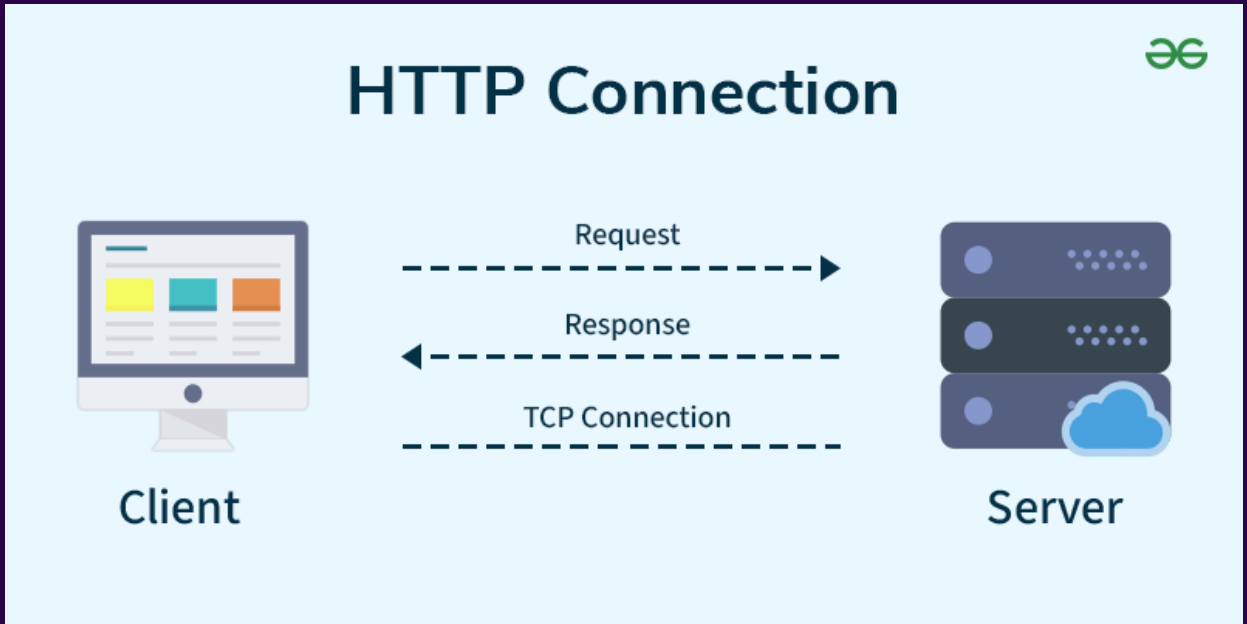
Multimedia Web Design - Exercises

HTTP REQUESTS AND RESPONSES

limani.lorik@ibu.edu.mk

The HTTP (Hypertext Transfer Protocol) is the foundation of communication on the web. It follows a request-response model where a client (e.g., browser or curl) sends a request to a server, and the server responds.



HTTP REQUEST

When a **client (browser)** talks to a **server**, it sends an **HTTP request**.
That request has **3 main parts:**
Request Line
This is the first line of the request. It tells the server what you want.
It contains:

- HTTP Method – what action to do
- Resource Path – what you want to access
- HTTP Version – rules of communication

Example: GET /index.html HTTP/1.1
Meaning:
GET → "I want to get something"
/index.html → "this specific page/file"
HTTP/1.1 → "using this HTTP version"

Common methods:
GET → get data
POST → send data
PUT → update data
DELETE → delete dat

HEADERS

Headers send extra information (metadata) about the request.
They help the server understand:
who is making the request
what kind of data is being sent
how to handle the request

Example headers:

Host: www.example.com
User-Agent: Chrome/120.0
Content-Type: application/json

Meaning:

- **Host** → which website/server
- **User-Agent** → browser or app used
- **Content-Type** → format of the data (JSON, HTML, form data, etc.)

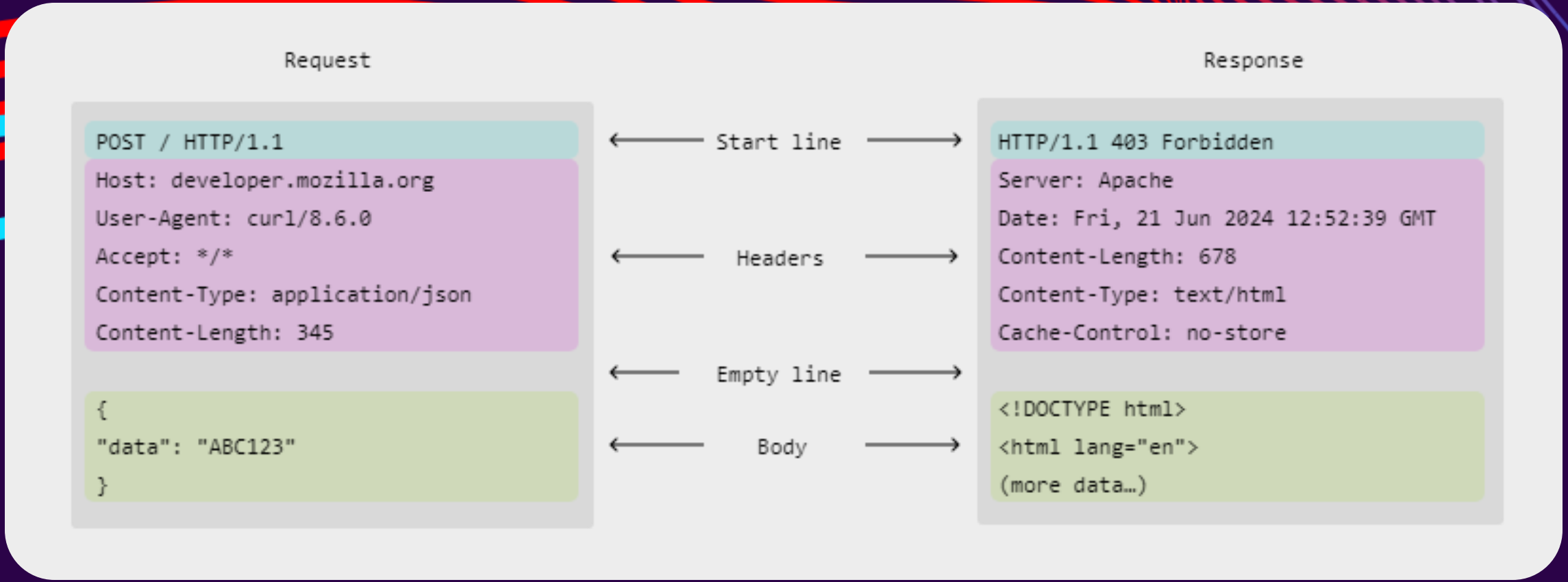
Headers are like **instructions on the food order**
(“no onions”, “extra spicy”, etc.)

HTTP RESPONSE COMPONENTS:

Status Line: Includes the HTTP version, status code (e.g., 200, 404), and reason phrase.

Headers: Metadata about the response, like Content-Type and Content-Length.

Body: The content returned by the server, such as HTML or JSON.



1.A *start-line* is a single line that describes the HTTP version along with the request method or the outcome of the request.
2.An optional set of *HTTP headers* containing metadata that describes the message. For example, a request for a resource might include the allowed formats of that resource, while the response might include headers to indicate the actual format returned.
3.An empty line indicating the metadata of the message is complete.
4.An optional *body* containing data associated with the message. This might be POST data to send to the server in a request, or some resource returned to the client in a response. Whether a message contains a body or not is determined by the start-line and HTTP headers.

WHAT IS CURL?

cURL (Client URL) is a command-line tool used to transfer data with URLs. It supports multiple protocols like HTTP, HTTPS, FTP, and more.

Used in Linux, macOS, Windows (via CMD/PowerShell).

Useful for automation and scripting.

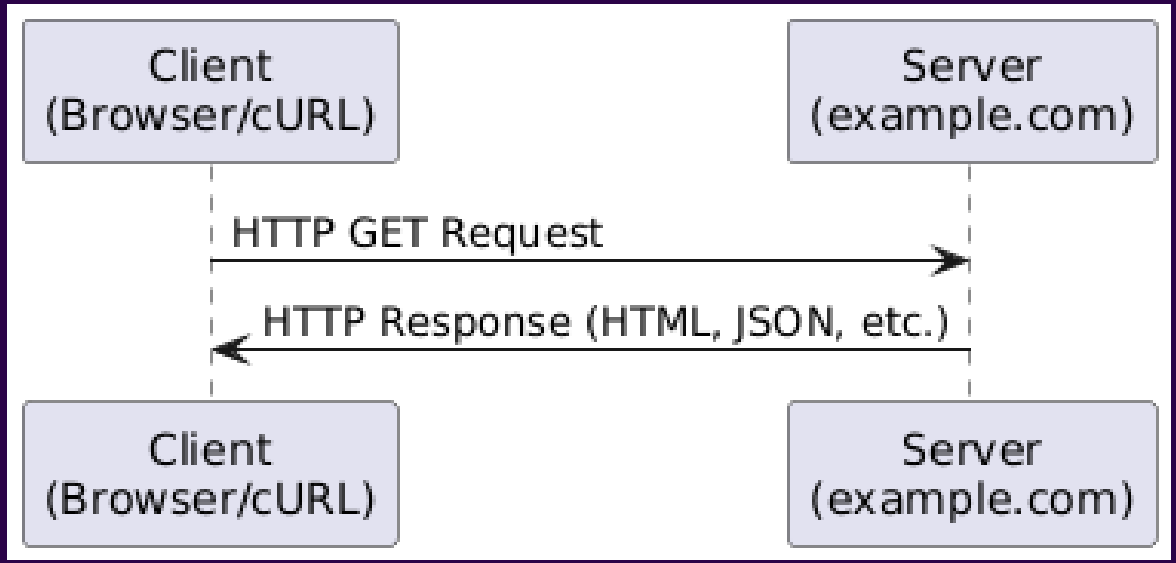
HTTP GET REQUEST

- curl https://example.com
- This command fetches the HTML content of example.com.

  Works like a browser requesting a webpage.

What Happens When We Run cURL?

- *The command sends an HTTP GET request.*
- *The server responds with an HTML document (or other data).*
- *You can see the raw webpage source in the terminal.*



GET REQUEST – FETCHING DATA

A **GET** request retrieves data from a server. No body is required.

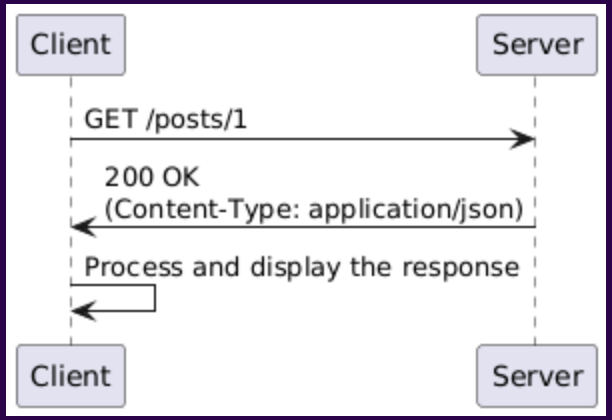
Example:
curl -X GET https://jsonplaceholder.typicode.com/posts/1

Explanation:

- Curl → Calls the cURL tool.
- -X GET → Specifies the request method (optional because GET is default).
- https://jsonplaceholder.typicode.com/posts/1 → API endpoint (fake JSON API for testing).

Expected Response (JSON)

```
{
  "userId": 1,
  "id": 1,
  "title": "Sample Title",
  "body": "Sample Body"
}
```



GET REQUEST WITH RESPONSE HEADERS

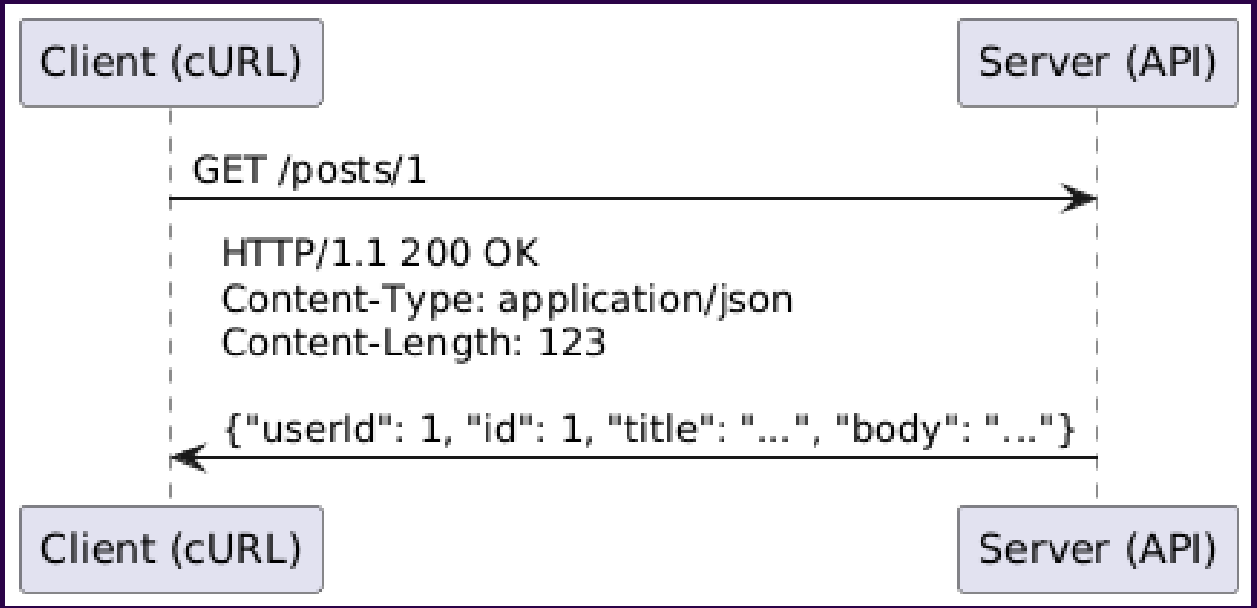
Example:
curl -i https://jsonplaceholder.typicode.com/posts/1

Explanation:

- -i → Includes the response headers.

Expected Response (JSON)

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 57
...
{
  "userId": 1,
  "id": 1,
  "title": "Sample Title",
  "body": "Sample Body"
}
```

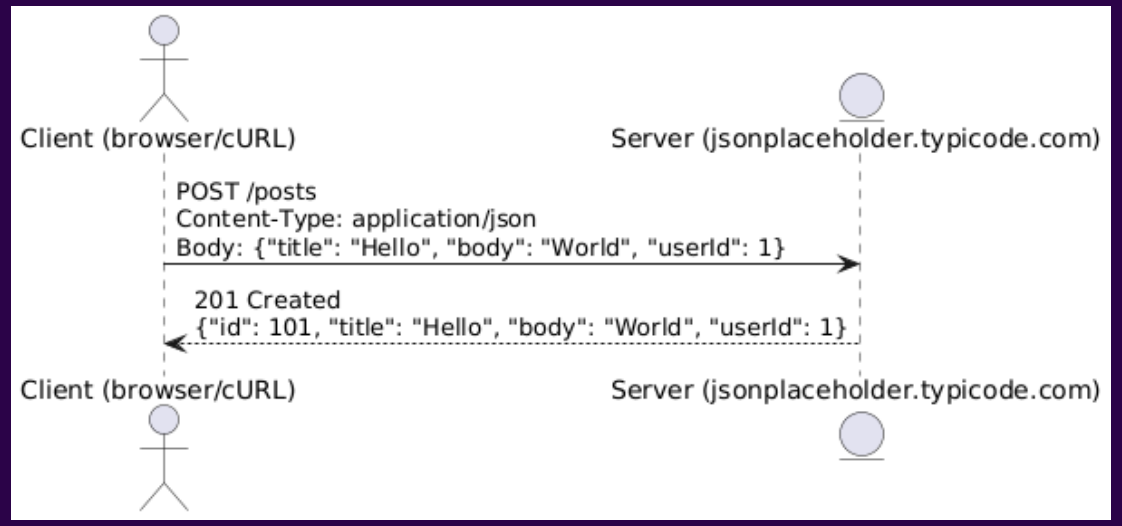



POST REQUEST – SENDING DATA

A POST request is used to send data to the server.

Example:

```
curl -X POST https://jsonplaceholder.typicode.com/posts \    -
H "Content-Type: application/json" \    -d '{"title": "Hello",
"body": "World", "userId": 1}'
```



Explanation:

- -X POST → Specifies POST method.
- -H "Content-Type: application/json" → Sets header to tell the server we're sending JSON.
- -d '{"title": "Hello", "body": "World", "userId": 1}' → The -d flag sends data in JSON

Expected Response (JSON)

```
{
  "title": "Hello",
  "body": "World",
  "userId": 1,
  "id": 101
}
```

The id: 101 is assigned by the API.

PUT REQUEST – UPDATING DATA

A PUT request updates existing data.

Example:

```
curl -X PUT https://jsonplaceholder.typicode.com/posts/1 \
    -H "Content-Type: application/json" \
    -d '{"title": "Updated Title", "body": "Updated Body", "userId": 1}'
```

Explanation:

- -X PUT → Specifies PUT method (used for updates).
- -H "Content-Type: application/json" → Defines the request content type.
- -d '{"title": "Updated Title", "body": "Updated Body", "userId": 1}' → The new updated data.

Expected Response (JSON)

```
{
  "title": "Updated Title",
  "body": "Updated Body",
  "userId": 1,
  "id": 1
}
```

DELETE REQUEST – REMOVING DATA

A DELETE request removes a resource.

Example:
curl -X DELETE https://jsonplaceholder.typicode.com/posts/1

Explanation:

- -X DELETE → Specifies DELETE method.

Expected Response (JSON)

{}

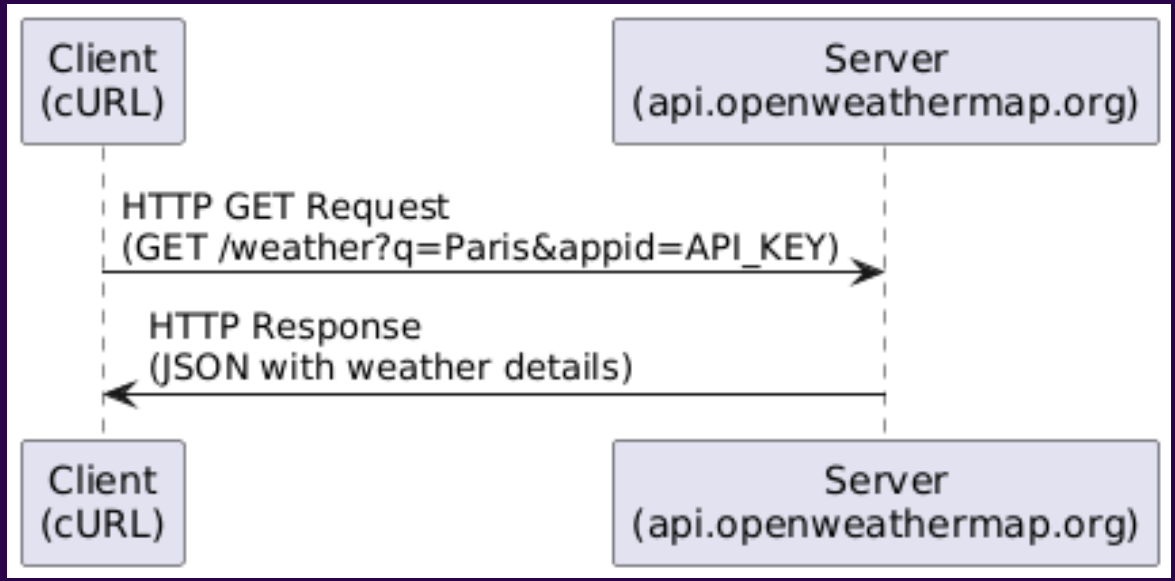
(The server removes the resource and returns an empty response.)

OPENWEATHER API (GET WEATHER DATA)

Use cURL to Fetch Weather Data

Curl

"https://api.openweathermap.org/data/2.5/weather?q=Paris&appid=38ed78e641edb9e0f9952a801601e729"



response received:


```
{
  "coord": {
    "lon": 2.3488,
    "lat": 48.8534
  },
  "weather": [
    {
      "id": 500,
      "main": "Rain",
      "description": "light rain",
      "icon": "10d"
    }
  ],
  "base": "stations",
  "main": {
    "temp": 283.14,
    "feels_like": 280.87,
    "temp_min": 282.58,
    "temp_max": 283.92,
    "pressure": 1023,
    "humidity": 92,
    "sea_level": 1023,
    "grnd_level": 1013
  },
  "visibility": 10000,
  "wind": {
    "speed": 4.63,
    "deg": 200
  },
  "rain": {
    "1h": 0.14
  },
  "clouds": {
    "all": 100
  },
  "dt": 1740046703,
  "sys": {
    "type": 2,
    "id": 2012208,
    "country": "FR",
    "sunrise": 1740034197,
    "sunset": 1740071942
  },
  "timezone": 3600,
  "id": 2988507,
  "name": "Paris",
  "cod": 200
}
```

"FETCHING WEATHER DATA FOR NEW YORK USING CURL AND OPENWEATHER API"

Use cURL to Fetch Weather Data

```
curl
"https://api.openweathermap.org/data/2.5/weather?q=New%20York&appid=38ed78e641edb9e0f9952a801601e729&units=metric"
```

JSON Response from OpenWeather API for New York

{"coord":{"lon":-74.006,"lat":40.7143},"weather":[{"id":800,"main":"Clear","description":"clear sky","icon":"01n"}],"base":"stations","main":{"temp":-6.97,"feels_like":-13.36,"temp_min":-8.84,"temp_max":-5.99,"pressure":1017,"humidity":64,"sea_level":1017,"grnd_level":1015},"visibility":10000,"wind":{"speed":4.63,"deg":350},"clouds":{"all":0},"dt":1740048631,"sys":{"type":1,"id":4610,"country":"US","sunrise":1740051786,"sunset":1740091001},"timezone":-18000,"id":5128581,"name":"New York","cod":200}

```
curl -H "User-Agent: MyCustomAgent" "https://httpbin.org/get"
```

This command is making an HTTP GET request to https://httpbin.org/get, but with a custom User-Agent header.

curl → The command-line tool for making HTTP requests.

-H "User-Agent: MyCustomAgent" → Sets a custom User-Agent header.

- Normally, when you send a request, your browser or cURL has a default User-Agent (e.g., curl/7.68.0).
- Here, we are overriding it to "MyCustomAgent".

"https://httpbin.org/get" → This is the API endpoint that simply returns whatever request details were sent.


```
{
  "args": {},
  "headers": {
    "User-Agent": "MyCustomAgent",
    "Host": "httpbin.org",
    "Accept": "*/*"
  },
  "origin": "192.168.1.10",
  "url": "https://httpbin.org/get"
}
```

SUBMITTING LOGIN CREDENTIALS WITH CURL (POST REQUEST)

For this example, we will use **ReqRes**, a free API for testing:

1. **Endpoint:** https://reqres.in/api/login
2. **Method:** POST
3. **Data Format:** JSON

```
curl -X POST "https://reqres.in/api/login" ^
    -H "Content-Type: application/json" ^
    -d "{\"email\": \"eve.holt@reqres.in\", \"password\": \"cityslicka\"}"
```

-X POST → Use **POST** method.
-H "Content-Type: application/json" → Set **request header** to JSON.
-d "{\"email\": \"eve.holt@reqres.in\", \"password\": \"cityslicka\"}" → Send login **data** in JSON format.

Response:

```
{
  "token": "QpwL5tke4Pnpja7X4"
}
```

USE THE TOKEN FOR AUTHENTICATION

```
curl -H "Authorization: Bearer QpwL5tke4Pnpja7X4" "https://reqres.in/api/users/2"
```

This sends a **GET request** with your **token** for authentication.

Instead of copying the token manually, you can **store it in a variable**:

```
FOR /F "tokens=2 delims=: " %a IN ('curl -s -X POST "https://reqres.in/api/login" -H "Content-Type: application/json" -d
"{\"email\": \"eve.holt@reqres.in\", \"password\": \"cityslicka\"}"') DO @SET TOKEN=%a
curl -H "Authorization: Bearer %TOKEN%" "https://reqres.in/api/users/2"
```

```
CURL -S -X POST
"HTTPS://REQRES.IN/API/LOGIN" ^
-H "CONTENT-TYPE: APPLICATION/JSON" ^
-D "{\"EMAIL\": \"EVE.HOLT@REQRES.IN\",
\"PASSWORD\": \"CITYSLICKA\"}"
```

curl -s → Runs cURL in **silent mode** (hides progress bar).
-X POST → Specifies **HTTP POST** request.
-H "Content-Type: application/json" → **Header** defining the request body format.
-d "{...}" → **Data payload** (email & password for login).

This request returns:

{ "token": "QpwL5tke4Pnpja7X4" }

```
FOR /F "tokens=2 delims=: " %a IN ('curl -s -X POST ...') DO
@SET TOKEN=%a
```

FOR /F → Loops through command output.
"tokens=2 delims=: " → Splits response using colon (:) and space (" "), taking the second part (the token).
%a → Stores the extracted value (i.e., "QpwL5tke4Pnpja7X4").
DO @SET TOKEN=%a → Saves the token into an environment variable %TOKEN%.

After running this, %TOKEN% will contain "QpwL5tke4Pnpja7X4".

```
curl -H "Authorization: Bearer %TOKEN%" "https://reqres.in/api/users/2"
```

-H "Authorization: Bearer %TOKEN%" → Adds the **Bearer token** (%TOKEN%) to the request.
"https://reqres.in/api/users/2" → Retrieves details for **User ID 2**.